Programming Language Reference Database

CSC 436 Final Project Report

Group Members

Matthew Barbrack

Matthew Petrarca

Hudson Byers

Ryan Megna

Victor Palino

1. Application Overview

The Programming Language Reference Database is a web based reference tool that lets users look up syntax, functions, operators, and data structures across multiple programming languages in one place. The application is deployed at mbarbrack.rhody.dev/Programming_Language_Lookup/ and is backed by a MySQL database hosted on Cpanel.

What the application is

A read accessible reference database that consolidates programming language information including functions, operators, and data structures, along with language specific syntax, examples, results, and notes for each implementation.

What problem it addresses

Developers and students frequently need to reference language specific syntax, especially when working across multiple languages. Instead of searching separate documentation sites for each language, this application provides a unified interface to browse and compare language features side by side.

Target audience

Computer science students, developers learning new languages, and anyone who regularly works across multiple programming languages and needs quick access to syntax references.

Key features

- Browse functions, operators, and data structures filtered by language, category, and sort order
- Side by side comparison of two languages for any selected function, operator, or data structure
- Admin panel allowing authenticated users to insert new languages, functions, operators, data structures, and implementations
- Role based database access with a read only public user and a write enabled admin user

2. Database Design

The database was designed using an Entity Relationship model centered on the concept of a programming language and its various implementations of functions, operators, and data structures.

Major entities

- language: stores the name and version of each programming language
- function_table: stores function names, categories, and descriptions independent of any specific language
- operator: stores operator names, symbols, categories, and descriptions
- data_structure: stores data structure names, categories, and descriptions
- implementation: stores the language specific details for how a function, operator, or data structure is used, including syntax, example, result, notes, and date added

Junction tables

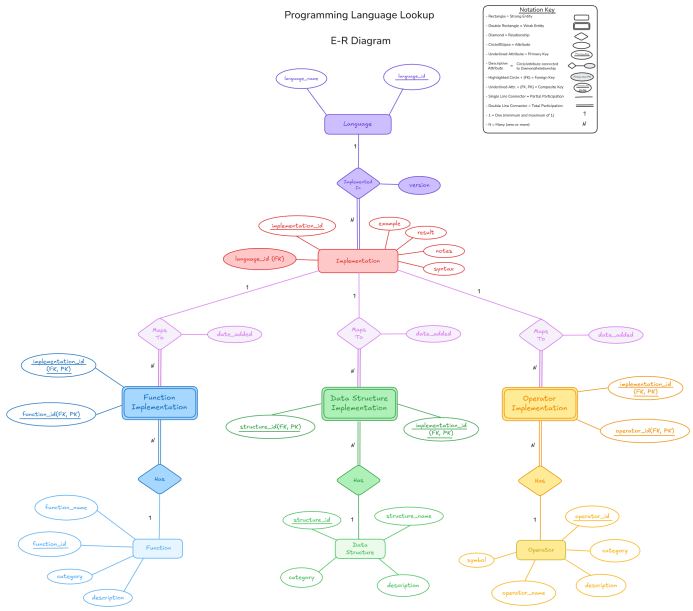
- function_implementation: links implementation rows to function_table rows
- operator_implementation: links implementation rows to operator rows
- structure_implementation: links implementation rows to data_structure rows

Primary keys and relationships

Each core table uses a single auto increment integer primary key. The implementation table references language through a foreign key on language_id. The three junction tables use composite primary keys combining implementation_id with the relevant entity ID, enforcing that no duplicate pairings can exist.

Important constraints

Foreign key constraints enforce referential integrity across all relationships. NOT NULL constraints apply to name fields in all tables. Composite primary keys on junction tables enforce uniqueness of implementation to entity pairings.



The ER diagram illustrates how the implementation table acts as the central linking table between a language and its specific usage of a function, operator, or data structure. This design avoids redundancy by storing language-agnostic definitions once in the entity tables and language specific details in the implementation table.

3. Database Implementation

The database mbarbrac_CSC436_Project was implemented in MySQL and is hosted on Cpanel. The following tables and views make up the full schema.

language

```
CREATE TABLE language (
  language_id   INT AUTO_INCREMENT PRIMARY KEY,
  language_name VARCHAR(100) NOT NULL,
  version       VARCHAR(50)
);
```

function_table

```
CREATE TABLE function_table (
  function_id   INT AUTO_INCREMENT PRIMARY KEY,
  function_name VARCHAR(100) NOT NULL,
  category      VARCHAR(50),
  description   TEXT
);
```

operator

```
CREATE TABLE operator (
  operator_id   INT AUTO_INCREMENT PRIMARY KEY,
  operator_name VARCHAR(100) NOT NULL,
  symbol        VARCHAR(20),
  category      VARCHAR(50),
  description   TEXT
);
```

data_structure

```
CREATE TABLE data_structure (
  structure_id   INT AUTO_INCREMENT PRIMARY KEY,
  structure_name VARCHAR(100) NOT NULL,
  category       VARCHAR(50),
  description    TEXT
);
```

implementation

```
CREATE TABLE implementation (
  implementation_id INT AUTO_INCREMENT PRIMARY KEY,
  language_id       INT NOT NULL,
  syntax            TEXT,
  example           TEXT,
  result            TEXT,
  notes             TEXT,
  date_added        DATE,
  FOREIGN KEY (language_id) REFERENCES language(language_id)
);
```

Junction tables with composite primary keys

```
CREATE TABLE function_implementation (
  implementation_id INT NOT NULL,
  function_id       INT NOT NULL,
  PRIMARY KEY (implementation_id, function_id),
  FOREIGN KEY (implementation_id) REFERENCES
implementation(implementation_id),
  FOREIGN KEY (function_id)       REFERENCES function_table(function_id)
);
```

The operator_implementation and structure_implementation tables follow the same pattern using operator_id and structure_id respectively.

Lookup views

Three views were created to simplify common queries across the application:

```
vw_function_lookup  -> joins language, implementation,
function_implementation, function_table
vw_operator_lookup  -> joins language, implementation,
operator_implementation, operator
vw_structure_lookup -> joins language, implementation,
structure_implementation, data_structure
```

How data was populated

Data was entered manually through the admin panel built into the application. Languages, functions, operators, and data structures were inserted individually, with implementations linked to entities through the junction tables. The database contains approximately 55 implementation rows distributed across five languages: Python, JavaScript, C, C++, and Java.

Figures 1-3: Sample data from the language, implementation, and function_table tables

Language Table

language_id	language_name	version
1	Python	3.x
2	C++	C++11
3	Java	Java 8+
4	JavaScript	ES6+

Implementation Table

implementation_id	language_id	date_added	example	result	notes	syntax
1	1	2026-02-22	print("Score:", 95, sep=" ")	Score: 95	Supports multiple arguments with customizable sepa...	print(value, sep=" ", end="\n")
2	2	2026-02-22	std::cout << "Temperature: " << 72 << "F" << std::...	Temperature: 72F	Chain multiple values with << operator	std::cout << value << std::endl;
3	3	2026-02-22	System.out.println("Items in cart: " + 5);	Items in cart: 5	Use + for string concatenation	System.out.println(value);
4	4	2026-02-22	console.log(`User ${name} logged in`);	User John logged in	Template literals use backticks and ${} for interp...	console.log(value);

Function Table

function_id	function_name	category	description
1	print	I/O	Outputs text or values to standard output
2	input	I/O	Reads input from standard input
3	len	Utility	Returns the length of a collection or string
4	sort	Sorting	Sorts elements in a collection
5	map	Functional	Applies a function to each element

4. Normalization

The database satisfies Third Normal Form across all tables.

1NF

All attribute values are atomic with no repeating groups or multi valued attributes. Every row is uniquely identifiable by its primary key.

2NF

All non key attributes in tables with single column primary keys are fully functionally dependent on that primary key. Junction tables have composite primary keys but no additional non key attributes, so 2NF is satisfied.

3NF and the version column correction

During development, a transitive dependency was identified and corrected. The version column had originally been placed in the implementation table, creating the dependency chain: implementation_id determines language_id, and language_id determines version. Since version depends on language_id and not directly on implementation_id, this was a 3NF violation.

The fix was to move the version to the language table, where it correctly depends only on language_id as the primary key. The three lookup views were updated to retrieve versions through a JOIN on the language table, preserving the same external interface while eliminating the transitive dependency.

Additional design decisions supporting normalization

- Language agnostic definitions are stored once in their own entity tables and not duplicated in the implementation table
- The implementation table stores only language-specific details, keeping concerns separated
- Junction tables handle many-to-many relationships without embedding multi-valued foreign keys

Originally the version column was placed in the implementation table. After identifying the transitive dependency it was moved to the language table, where it correctly depends on language_id as the primary key.

5. Application Functionality and Database Integration

5a. SQL Queries

Query 1: Retrieve all functions for a given language

Purpose: Powers the main browse page when a user filters by language and function type. This is one of the most frequently executed queries in the application.

```
SELECT language_name, function_name, category, description,
```

```
        syntax, example, result, notes
FROM vw_function_lookup
WHERE language_name = 'Python'
ORDER BY category, function_name;
```

The view encapsulates the multi table join so the PHP layer stays simple while the view handles joining language, implementation, function_implementation, and function_table.

Query 1:

language_name	function_name ▲ 2	category ▲ 1	description	syntax	example	result	notes
Python	filter	Functional	Filters elements based on a condition	filter(function, iterable)	ages = [12, 18, 25, 8, 30] adults = list(filter(la...	[18, 25, 30]	Returns elements where function returns True
Python	map	Functional	Applies a function to each element	map(function, iterable)	nums = [1, 2, 3, 4] squared = list(map(lambda x: x...	[1, 4, 9, 16]	Returns iterator; wrap in list() to see results
Python	input	I/O	Reads input from standard input	variable = input(prompt)	age = int(input("Enter your age: "))	User enters: 25 -> age = 25	Always returns string; cast to int/float as needed
Python	print	I/O	Outputs text or values to standard output	print(value, sep=" ", end="\n")	print("Score:", 95, sep=" ")	Score: 95	Supports multiple arguments with customizable sepa...
Python	sort	Sorting	Sorts elements in a collection	list.sort() or sorted(iterable)	prices = [29.99, 9.99, 49.99, 19.99] prices.sort()	[9.99, 19.99, 29.99, 49.99]	.sort() modifies in place; sorted() returns new li...
Python	join	String	Joins elements into a string	separator.join(iterable)	words = ["Hello", "World", "2024"] " ".join(words)	Hello World 2024	Separator is the string you call join on
Python	split	String	Splits a string into an array/list	string.split(separator)	csv_line = "John,25,Engineer" fields = csv_line.sp...	["John", "25", "Engineer"]	Default separator is whitespace
Python	len	Utility	Returns the length of a collection or string	len(sequence)	words = ["apple", "banana", "cherry"] len(words)	3	Works on strings, lists, tuples, dicts, sets

Query 2: Side by side comparison of two languages for a specific operator

Purpose: Powers the Compare tab by returning both implementations in a single query so the frontend can render them as side by side cards.

```
SELECT language_name, operator_name, symbol, category, syntax, example,
result, notes
FROM vw_operator_lookup
WHERE language_name IN ('Python', 'Java')
AND symbol = '+'
ORDER BY language_name;
```

By filtering on language_name IN with two values, the application retrieves both implementations at once. The frontend splits the result rows into side by side cards.

Query 2:

language_name ▲ 1	operator_name	symbol	category	syntax	example	result	notes
Java	Addition	+	Arithmetic	a + b	int hoursWorked = 8 + 6 + 9;	23	String + int auto-converts int to String
Python	Addition	+	Arithmetic	a + b	total = 15.99 + 4.50 + 2.00	22.49	Also concatenates strings and lists

Query 3: Admin INSERT wrapped in a transaction

Purpose: Adds a new implementation and links it to a function in a single atomic operation. Both inserts are wrapped in a transaction so that if either fails, no partial data is committed.

```
Step 1: Insert the implementation
```

```
INSERT INTO implementation (language_id, syntax, example, result, notes,
date_added)
VALUES (?, ?, ?, ?, ?, CURDATE());
```

1 row inserted.
Inserted row id: 60 (Query took 0.0003 seconds.)

```
INSERT INTO implementation (language_id, syntax, example, result, notes, date_added) VALUES (4, 'a + b', 'x = 5 + 3', '8', 'Also concatenates strings',
CURDATE());
```

[Edit inline] [Edit] [Create PHP code]

```
Step 2: Link to the function
INSERT INTO function_implementation (implementation_id, function_id)
VALUES (LAST_INSERT_ID(), ?);
```

1 row inserted. (Query took 0.0005 seconds.)

```
INSERT INTO operator_implementation (implementation_id, operator_id) VALUES (60, 1);
```

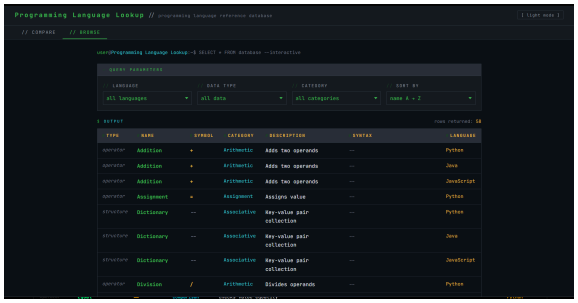
[Edit inline] [Edit] [Create PHP code]

5b. Application Interface

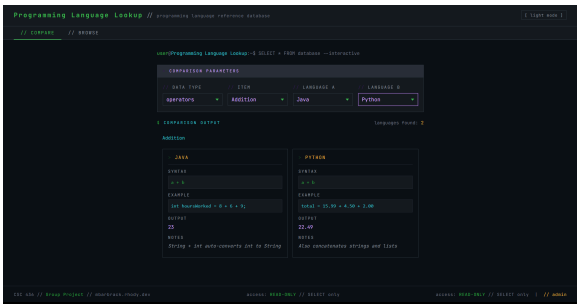
Public interface

- Top navigation with links to the Browse and Compare tabs
- Browse tab: four dropdowns for language, data type, category, and sort order that filter results rendered as terminal style output cards
- Compare tab: two language selectors and an item selector that render side by side implementation cards

Browse Tab:



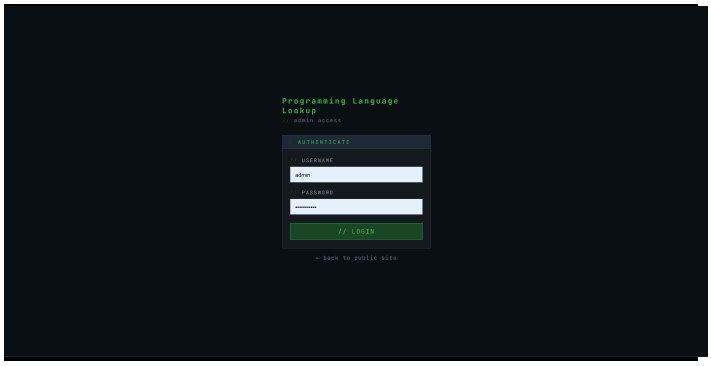
Compare Tab:



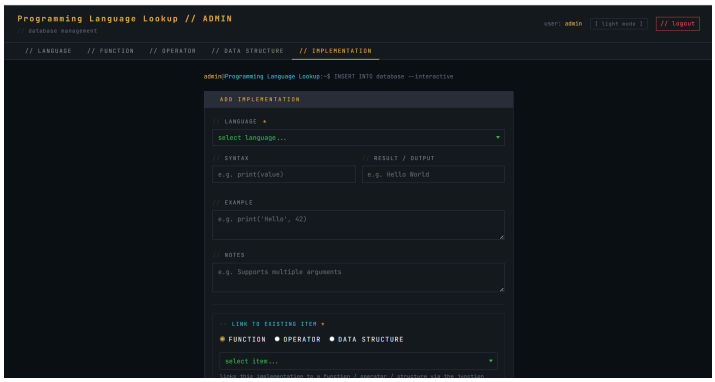
Admin interface

- Login page at /Programming_Language_Lookup/admin/login.php requiring a username and password
- Admin dashboard with tabbed sections for adding languages, functions, operators, data structures, and implementations
- Dropdowns in the implementation tab are populated dynamically from the database

Login Page:



Insert Implementation Tab:



5c. Frontend and Backend Integration

The application uses a three tier architecture with HTML, CSS, and JavaScript on the frontend, PHP on the backend, and MySQL as the database layer.

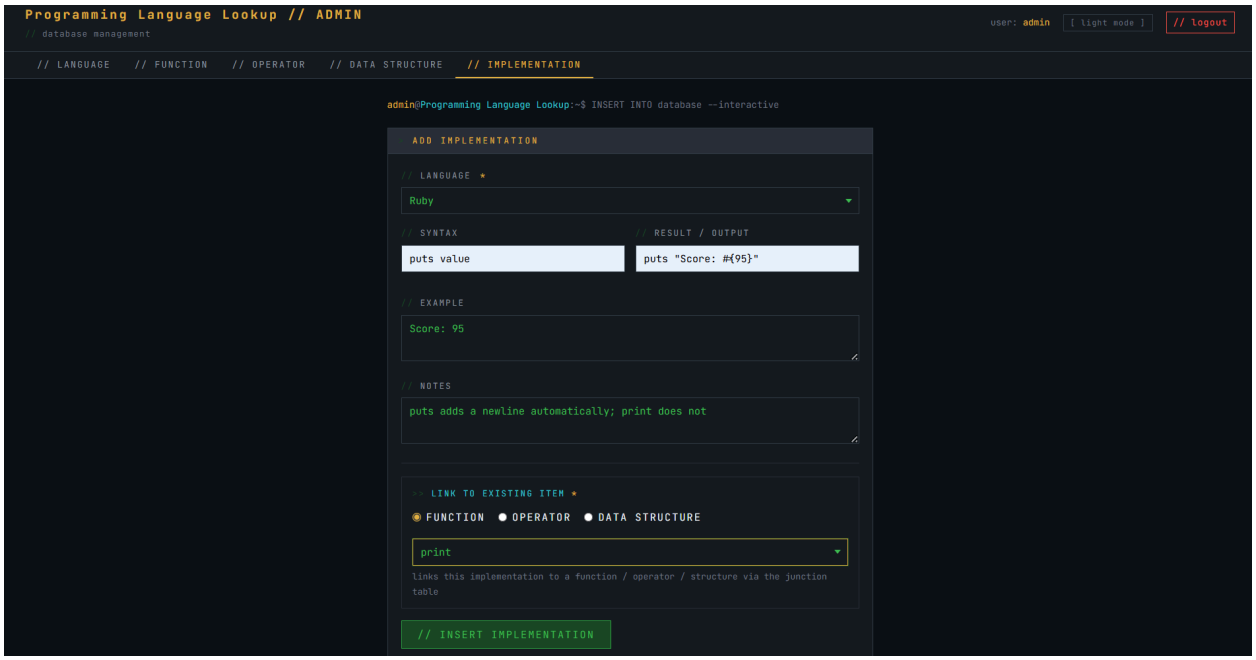
Data retrieval on the public side

The application uses HTML form GET requests to filter results. When a user changes a dropdown, the page reloads with the selected values as URL parameters, which PHP reads to run the appropriate query and render the results.

Data insertion on the admin side

The admin panel uses HTML forms that POST to the same PHP page. PHP validates form values server-side and executes prepared INSERT statements via PDO. Multi-step inserts are wrapped in a transaction.

Example Implementation:



☐	Edit Copy Delete	61	6	2026-05-04	Score: 95	puts "Score: #{95}"	puts adds a newline automatically; print does not	puts value

Technologies used

- Frontend: HTML5, CSS3
- Backend: PHP
- Database: MySQL on Cpanel
- Local development: XAMPP
- Deployment: cPanel File Manager to public_html/Programming_Language_Lookup/

6. Security, Privacy, and Data Protection

Prepared statements for SQL injection prevention

All database queries use PDO prepared statements with bound parameters, completely separating user supplied input from SQL logic. PDO is configured with

ATTR_EMULATE_PREPARES set to false, forcing true server side prepared statements rather than client-side emulation.

```
$stmt = $pdo->prepare(
  'INSERT INTO language (language_name, version) VALUES (:name,
:version)'
);
$stmt->execute([':name' => $name, ':version' => $version]);
```

Output sanitization for XSS prevention

All data retrieved from the database and rendered in the browser is passed through html special chars before output. This converts special characters to their HTML entity equivalents and prevents injected scripts from executing.

```
echo htmlspecialchars($row['function_name'], ENT_QUOTES, 'UTF-8');
```

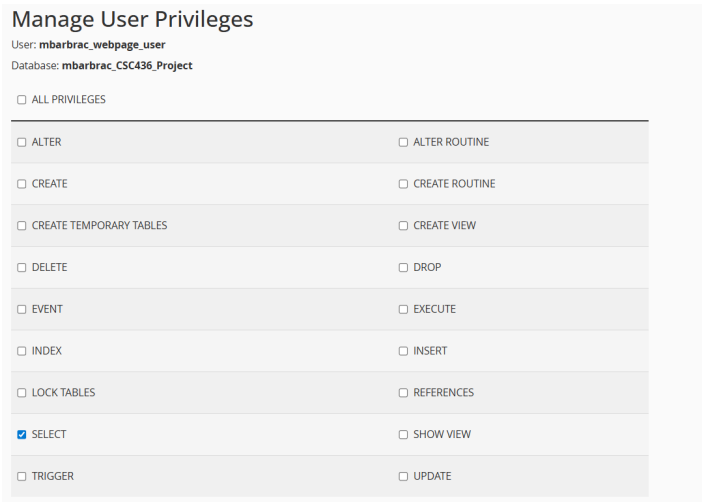
Admin authentication

The admin panel uses session based authentication. Admin credentials are stored in a .env file loaded at runtime with parse_ini_file. All admin pages check for an active session before rendering. Logout destroys the session completely.

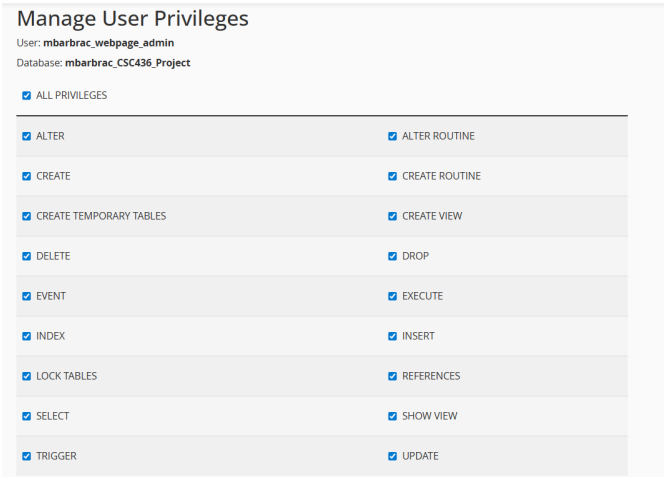
Role-based database access

Two separate MySQL users are configured. The public user mbarbrac_webpage_user has SELECT only privileges. The admin user mbarbrac_webpage_admin has INSERT privileges. This limits the impact of any credential exposure.

Base User Privileges:



Admin User Privileges:



ATTR_EMULATE_PREPARES set to false:

```php
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];
```

7. Optimization and Reliability Enhancements

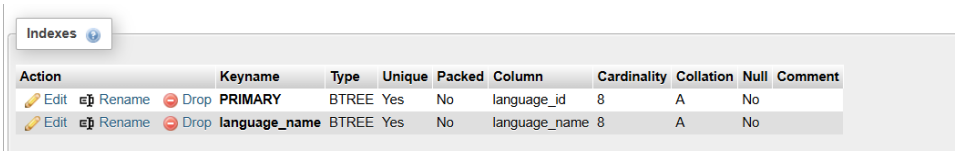
Indexed Columns

Indexes were added to columns that appear frequently in WHERE clauses and JOIN conditions. The language_name column is queried on nearly every page load, and language_id in the implementation table is used in every join between those two tables.

```
CREATE INDEX idx_language_name ON language (language_name);
CREATE INDEX idx_impl_language  ON implementation (language_id);
CREATE INDEX idx_func_name      ON function_table (function_name);
CREATE INDEX idx_op_name        ON operator (operator_name);
```

Without the index on language_name, MySQL would perform a full table scan on every browse or compare request. With the index, the engine locates matching rows directly, which becomes more important as the number of supported languages grows.

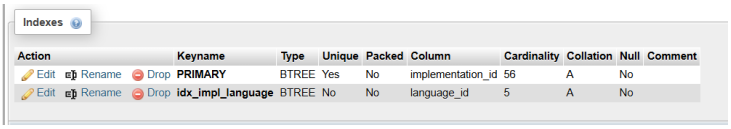
Language Table Indexes



Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	language_id	8	A	No	
Edit Rename Drop	language_name	BTREE	Yes	No	language_name	8	A	No	

Implementation Table Indexes



Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	implementation_id	56	A	No	
Edit Rename Drop	idx_impl_language	BTREE	No	No	language_id	5	A	No	

Optimized Query

The primary browse query originally joined four tables directly on each request. The join logic was encapsulated in three lookup views, reducing the PHP query to a simple SELECT with a WHERE clause against the view.

The query was analyzed using EXPLAIN FORMAT=JSON, which confirmed that MySQL uses the index on language_id in the implementation table during the join, avoiding a full scan of implementation rows. The join order starts from the language filter and traverses outward, which is the most efficient path.

```
SELECT * FROM vw_function_lookup
WHERE language_name = ? AND category = ?
ORDER BY function_name;
```

EXPLAIN output showing index usage across the vw_function_lookup join

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	l	*NULL*	const	PRIMARY,language_name	language_name	302	const	1	100.00	Using filesort
1	SIMPLE	f	*NULL*	ALL	PRIMARY	*NULL*	*NULL*	*NULL*	10	10.00	Using where
1	SIMPLE	fi	*NULL*	ref	PRIMARY,idx_func_impl_function	idx_func_impl_function	4	mbarbrac_CSC436_Project.f.function_id	2	100.00	Using index
1	SIMPLE	i	*NULL*	eq_ref	PRIMARY,idx_impl_language	PRIMARY	4	mbarbrac_CSC436_Project.fi.implementation_id	1	43.10	Using where

Transactions

The admin insert workflow involves two sequential writes: one to the implementation table and one to the appropriate junction table. These are wrapped in a transaction to preserve data integrity.

```
$pdo->beginTransaction();
try {
  $stmt->execute([$langId, $syntax, $example, $result, $notes]);
  $implId = $pdo->lastInsertId();
  $stmt2->execute([$implId, $funcId]);
  $pdo->commit();
} catch (Exception $e) {
  $pdo->rollBack();
}
```

If either insert fails, the rollback ensures no partial data is committed, preventing orphaned implementation rows with no linked function, operator, or data structure.

Admin insert success after adding a new implementation

implementation_id	language_id	date_added	example	result	notes	syntax
60	4	2026-05-04	x = 5 + 3	8	Also concatenates strings	a + b
61	6	2026-05-04	Score: 95	puts "Score: #{95}"	puts adds a newline automatically; print does not	puts value

phpMyAdmin confirming the new rows in the implementation

implementation_id	function_id
61	1

8. Reflection and Final Improvements

Strengths

Feedback from the Database Expo was positive. Reviewers highlighted the forward-thinking extensibility of the design, particularly storing language-agnostic definitions in separate tables and linking them through the implementation table. This makes it straightforward to add new languages without modifying the core schema. The consistent use of foreign keys and referential integrity constraints was also recognized as a strength.

The early decision to separate concerns between the entity tables and the implementation table paid off during development by keeping the PHP query logic much simpler than a flat schema would have required.

Revisions and Improvements

The primary structural revision was correcting the placement of the version column. Originally stored in the implementation table, it was identified as a transitive dependency and moved to the language table. The three lookup views were updated accordingly.

The second structural revision was updating the junction tables to composite primary keys. The original design used implementation_id alone, which limited each implementation to linking to only one entity. Composite keys correctly reflect the many-to-many design intent and make the schema more scalable.

On the application side, the admin panel was built to provide a meaningful write interface through the web application itself, demonstrating full CRUD integration between the frontend and backend.

Team Contributions

- **Matt Barbrack:** Frontend development, PHP backend integration, admin panel, deployment, cPanel configuration, and security implementation.
- **Mathew Petrarca:** Designed and documented original, updated, and final E-R Diagram. Ensured 3NF compliance, fundamental database scalability, and solved

core redundancy issues in original database design for seamless database transferability from design to live SQL implementation.

- **Ryan Megna:** CLI tool integration, made it so you can connect the tool to the database to keep updates live within the tool, used for quick lookups for documentation and comparisons between languages
- **Hudson Byers:** Collected a range of relevant data to populate the database by compiling csv, comma separated value, files for implementation of functions, data structures, and operators across multiple programming languages.
- **Victor Paulino:** Optimized database performance through indexing and query optimization on the language and implementation tables. Improved search efficiency for fast and reliable retrieval of the syntax and/or details of functions, structures, and operators.

Lessons Learned

One of the biggest challenges was deployment. The differences between XAMPP and cPanel in file paths, database host addresses, and username formatting caused issues that required careful debugging. The lesson was that deployment details need to be treated as a first-class concern from the beginning of the project.

The normalization correction reinforced how important it is to trace every attribute back to the primary key of its own table before finalizing a schema. The version issue would have been easier to catch with a more deliberate normalization review earlier in the process.

Implementing prepared statements and role-based database access highlighted how much of web application security is built into the data access layer rather than the frontend.

Future Improvements

- Add user registration and login so users can save favorite language comparisons
- Expand the dataset to cover more languages and a broader set of functions and operators
- Add full text search across function names, descriptions, and syntax
- Implement pagination on the browse page for large result sets
- Add an audit log table to record admin inserts with timestamps and usernames
- Deploy with HTTPS and enforce secure cookie settings on admin sessions

- Keep new page for old screen shot